

Department of Computer and Software Engineering- ITU

CE101L: Object Oriented Programming Lab

Course Instructor: Nadir Abbas	Dated: 04/03/2025
Teaching Assistant: Zainab, Nahal, Bilal	Semester: Spring 2025
Session: 2024-2028	Batch: BSCE24

Lab 7. Understanding Class Relationships – Association, Aggregation, and Composition

Name	Roll number	Obtained Marks/35

Checked on: _____

Signature: _____

Objective

The objective of this lab is to help students understand and apply concepts of Inheritance.

Equipment and Component

Component Description	Value	Quantity
Computer	Available in lab	1

Conduct of Lab

1. Students are required to perform this experiment individually.
2. In case the lab experiment is not understood, the students are advised to seek help from the course instructor, lab engineers, assigned teaching assistants (TA) and lab attendants.

Theory and Background

In Object-Oriented Programming (OOP), **relationships between classes** define how objects interact with each other. The key relationships covered in this lab are **Association, Aggregation, and Composition**.

1. Aggregation

Aggregation is a "**has-a**" relationship where one class contains an object of another class, but the contained object **can exist independently**.

- ♦ **Example:** A **Car** has an **Engine**, but the **Engine** can exist separately and be used in other cars.

2. Composition

Composition is a **strong form of Aggregation**, where one class **owns** objects of another class, and those objects **cannot exist independently**.

- ♦ **Example:** A **Building** consists of multiple **Rooms**, but a **Room** cannot exist without a **Building**.

These concepts help in designing modular, reusable, and well-structured object-oriented systems.

Tasks

Task 1: Accept the assignment posted in Google Classroom and after accepting clone the repository to your computer for this ensure you have logged into github app with your account.

Task 2: Solve the given problems written after task instructions, write code through IDE like CLion

Task 3: Ensure your code/solution is in the cloned folder.

Task 4: Commit and Push the changes through the Github App

Write code in functions, in files **functions.cpp**, **functions.h** and **main.cpp** after completing each part, verify through running code using “**make run**” on Cygwin.

Q1: Aggregation (Student and Library Example)

Write a program demonstrating Association where a Library checks if a Student has borrowed a book. A **Library has Students**, but a **Student can exist without a Library**

Class Design:

- **Class Student**
 - Private Members:
 - int id (Student ID)
 - string name (Student Name)
 - Library* library; (Aggregation)
 - Public Functions:
 - Constructor default
 - void setId(int); & int getId();
 - void setName(string); & string getName();
 - void setLibrary(Library*);
 - bool canBorrowBook(); → Returns true if student is eligible and book is available
 - bool isEligibleForLibrary(); → Returns true if ID is even
- **Class Library**
 - Private Members:
 - bool isBorrowed (Book Borrowed Status)
 - Public Functions:
 - Constructor default
 - void setIsBorrowed(bool); & bool getIsBorrowed();

Q2: Aggregation (Car and Engine Example)

Write a program demonstrating Aggregation, where a Car has an Engine object, but an Engine can exist separately.

Class Design:

- **Class Engine**
 - Private Members:

- string engineType
- Public Functions:
 - Constructor default
 - void setEngineType(string); & string getEngineType();
 - bool isHighPerformance(); → Returns true if engine type is "V8" or "Turbo"
- **Class Car**
 - Private Members:
 - string carName
 - Engine* engine; (Aggregation)
 - Public Functions:
 - Constructor default
 - void setCarName(string); & string getCarName();
 - void setEngine(Engine*); & Engine* getEngine();
 - bool isSportsCar(); → Returns true if engine is high performance

Q3: Composition (Building and Room Example)

Write a program demonstrating Composition, where a Building has Room objects that cannot exist independently.

Class Design:

- **Class Room**
 - Private Members:
 - int roomNumber
 - Public Functions:
 - Constructor default
 - void setRoomNumber(int); & int getRoomNumber();
 - bool isEvenRoom(); → Returns true if room number is even
- **Class Building**
 - Private Members:
 - string buildingName
 - Room rooms[3]; (Fixed-size array of rooms)
 - Public Functions:
 - **Building(Room rooms[3]);** → Constructor should accept an array of Room objects
 - void setBuildingName(string); & string getBuildingName();
 - bool hasEvenRoom(); → Returns true if at least one room has an even number

Q4: Aggregation (Doctor and Patient Example)

Write a program demonstrating Association, where a Doctor treats multiple Patients, and a Patient visits multiple Doctors.

Class Design:

- **Class Doctor**
 - Private Members:

- string name
 - Patient* patients[10]; (Array of pointers to patients)
 - int patientCount;
- Public Functions:
 - Constructor default
 - void setName(string); & string getName();
 - void addPatient(Patient*);
 - bool hasPatient(string); → Returns true if the doctor has a patient with the given name
- **Class Patient**
 - Private Members:
 - string name
 - Doctor* doctors[5]; (Array of pointers to doctors)
 - int doctorCount;
 - Public Functions:
 - Constructor default
 - void setName(string); & string getName();
 - void addDoctor(Doctor*);
 - bool hasDoctor(string); → Returns true if the patient has a doctor with the given name

Q5: UML Diagram (Separate Question)

Create a UML Class Diagram for:

1. Association (Doctor and Patient)
2. Composition (Building and Room)
3. Aggregation (Car and Engine)

UML Diagram Requirements:

- Show class names, attributes, and functions.
- Use correct relationships
- Make all in a single diagram and save as **UML_Diagram.png**

Assessment Rubric for Lab

Performance metric	CLO	Able to complete the task over 80% (4-5)	Able to complete the task 50-80% (2-3)	Able to complete the task below 50% (0-1)	Marks
1. Realization of experiment	1	Executes without errors excellent user prompts, good use of symbols, spacing in output. Through testing has been completed.	Executes without errors, user prompts are understandable, minimum use of symbols or spacing in output. Some testing has been completed.	Does not execute due to syntax errors, runtime errors, user prompts are misleading or non-existent. No testing has been completed.	
2. Conducting experiment	1	Able to make changes and answered all questions.	Partially able to make changes and few incorrect answers.	Unable to make changes and answer all questions.	
3. Computer use	2	Document submission timely.	Document submission late.	Document submission not done.	
4. Teamwork	3	Actively engages and cooperates with other group member(s) in effective manner.	Cooperates with other group member(s) in a reasonable manner but conduct can be improved.	Distracts or discourages other group members from conducting the experiment	
5. Laboratory safety and disciplinary rules	3	Code comments are added and does help the reader to understand the code.	Code comments are added and does not help the reader to understand the code.	Code comments are not added.	
6. Data collection	3	Excellent use of white space, creatively organized work, excellent use of variables and constants, correct identifiers for constants, No line-wrap.	Includes name, and assignment, white space makes the program fairly easy to read. Title, organized work, good use of variables.	Poor use of white space (indentation, blank lines) making code hard to read, disorganized and messy.	
7. Data analysis	4	Solution is efficient, easy to understand, and maintain.	A logical solution that is easy to follow but it is not the most efficient.	A difficult and inefficient solution.	
Total (out of 35):					